RELATÓRIO DE AUDITORIA TÉCNICO-FUNCIONAL

Prova de conceito · Aplicação Consultório Privado

Data de referência: 21 de outubro de 2026 | Versão: 1.0 — consolidação das evidências disponíveis

Objeto: avaliação independente de uma aplicação demonstrativa ASP.NET Core MVC, Entity Framework Core e SQL Server, com dados fictícios.

Natureza do trabalho: auditoria e recomendações; sem alterações, correções ou refatoração da aplicação.

Consultora: Flórida Araújo

1. Resumo executivo

A avaliação combinou inspeção de código-fonte, comandos de análise estática .NET, pesquisa de dependências NuGet e um teste funcional documentado. A principal falha funcional reproduzida foi a possibilidade de agendar uma consulta para um médico sem horário configurado. Foram ainda identificados riscos relevantes de controlo de acesso, gestão de dependências, coerência de agendamentos e tratamento de exceções. A análise identificou também boas práticas, nomeadamente a utilização de validação antifalsificação nas 21 ações explicitamente identificadas como POST.

A aplicação é uma PoC local e não um sistema clínico em produção. As classificações refletem risco potencial no cenário de utilização real e não demonstram, por si só, exploração ou impacto ocorrido.

2. Âmbito, metodologia e critérios

Âmbito: controladores MVC, regras de negócio de consultas/horários/pagamentos, configuração de arranque, persistência, logging, dependências e atributos HTTP.

Métodos:

- leitura de código;
- análise de saída do dotnet build e de analisadores;
- pesquisa PowerShell;
- teste funcional TF-001.

Não foram realizados testes de intrusão nem alterações ao código.

Escala: Crítica (impacto potencial muito elevado); Alta (falha de negócio/segurança relevante); Média (fragilidade com impacto contextual); Baixa (manutenibilidade/consistência); A confirmar (evidência insuficiente).

3. Matriz de constatações

Ref.	Síntese	Avaliação	Base
AUD-003.1	Lógica e acesso a dados nos controladores	Média	Análise de código
AUD-003.2	Concorrência no agendamento	A confirmar	Hipótese; sem teste concorrente
AUD-003.3 / TF-001	Consulta sem horário médico	Alta	Teste funcional reproduzido

AUD-004	Ausência de autenticação/autorização aplicacional	Crítica se produção	Program.cs e controladores
AUD-005 / 018	Confiança no certificado SQL Server	Média	Configuração
AUD-006	Cobertura de testes e observabilidade	Média	Inventário/inspeção
AUD-007	Pagamento em consulta cancelada	Condicional	Análise de validações
AUD-008	Remoção de horário sem validar consultas futuras	Alta condicional	Código RemoverHorario
AUD-009	Atualizações concorrentes sem controlo identificado	A confirmar	Análise preliminar
AUD-010–012	Restrições e estados dependentes da aplicação	A confirmar	Análise preliminar
AUD-014.2	Dependência transitiva com avisos NuGet	Alta para triagem	NU1902/NU1903
AUD-015	Avisos de analisadores .NET	Mista	Rebuild AnalysisMode=All
AUD-016	Mensagens genéricas de erros de persistência	Média	8 blocos catch
AUD-017	Logging contextual não identificado	Média	Program.cs/appsettings e pesquisa
AUD-019	Destino de exceções /Home/Error não localizado	Média	Inspeção estática
AUD-020	Atributos de proteção CSRF em POST	Boa prática	21 de 21 POST
AUD-021	Operações de gravação e eliminação	Verificação	21 SaveChangesAsync

4. Constatações fundamentadas

AUD-003.3 / TF-001 — Agendamento sem horário médico

Classificação: Alta | Evidência: Teste funcional e método ValidarAgendamentoAsync

Observação: Foi registada com sucesso uma consulta fictícia (#2005) para o médico Bruno Teixeira em 22/10/2026, 10:15–10:45, sem horário configurado. A validação de horários está condicionada à existência de registos de horário.

Impacto: Possibilidade de marcações incompatíveis com a disponibilidade médica.

Recomendação: Exigir horário ativo compatível antes de permitir a marcação; testar ausência de horário e limites temporais.

Limite da evidência: Teste realizado em ambiente de demonstração; a evidência visual original deve acompanhar o anexo quando disponível.

AUD-004 — Controlo de acesso aplicacional

Classificação: Crítica se produção | Evidência: Program.cs e inspeção dos controladores

Observação: O arranque configura MVC e DbContext, mas não autenticação/autorização; não foram identificadas restrições de acesso aos controladores analisados.

Impacto: Exposição ou alteração não autorizada de dados de utentes, consultas e pagamentos caso publicado sem proteção adicional.

Recomendação: Definir identidade, perfis, políticas de autorização e testes de acesso negativo.

Limite da evidência: Não equivale a afirmar que a aplicação PoC esteja publicamente exposta.

AUD-008 — Remoção de horários e consultas futuras

Classificação: Alta condicional | Evidência: MedicosController.RemoverHorario

Observação: A ação POST protegida por antiforgery localiza o horário e chama Remove/SaveChangesAsync sem verificar consultas futuras.

Impacto: Inconsistência entre disponibilidade médica e marcações existentes.

Recomendação: Definir regra de negócio e impedir remoção ou exigir gestão explícita de consultas afetadas.

Limite da evidência: Não foi demonstrado que a remoção apague consultas; depende da regra aprovada.

AUD-007 — Validação de pagamentos por estado da consulta

Classificação: Condicional | Evidência: PagamentosController.ValidarAsync

Observação: A validação confirma existência da consulta, data não futura e referência não duplicada; não verifica o estado da consulta.

Impacto: Pode permitir pagamentos ligados a consultas canceladas, se proibidos pela regra de negócio.

Recomendação: Formalizar regras para cancelamentos, taxas e pré-pagamentos e testar os cenários.

Limite da evidência: Não classificar como defeito confirmado sem decisão funcional.

AUD-014.2 — Avisos de vulnerabilidade em dependência transitiva

Classificação: Alta para triagem | Evidência: NU1902 e NU1903

Observação: System.Security.Cryptography.Pkcs 9.0.11 gerou avisos moderado e alto, repetidos nos projetos Web e Infrastructure; cadeia observada: EF Core SQL Server → Microsoft.Data.SqlClient 6.1.6 → Pkcs 9.0.11.

Impacto: Exposição potencial a vulnerabilidades conhecidas conforme a utilização efetiva da biblioteca.

Recomendação: Verificar advisories, versões corrigidas, dependência efetivamente carregada e alcance da vulnerabilidade.

Limite da evidência: Dois avisos distintos repetidos em dois projetos; não são quatro vulnerabilidades nem há exploração demonstrada.

AUD-015 — Análise estática .NET

Classificação: Melhoria técnica | Evidência: Rebuild com AnalysisMode=All

Observação: A reconstrução apresentou 190 avisos, incluindo 4 ocorrências NuGet e 186 avisos CA: CA2007 (127), CA1062 (19), CA1515 (17), CA5395 (15), CA1002/CA2227 (4), CA1861 (2), CA1305 (1), CA1725 (1).

Impacto: Riscos heterogéneos de legibilidade, API e consistência; nem todos representam vulnerabilidades.

Recomendação: Triar por relevância e contexto ASP.NET Core; priorizar CA5395 em ações relevantes e não aplicar recomendações mecanicamente.

Limite da evidência: CA2007 e CA1515 exigem avaliação contextual; contagens referem ocorrências, não defeitos únicos.

AUD-016 — Exceções de persistência com diagnóstico impreciso

Classificação: Média | Evidência: 8 catch (DbUpdateException)

Observação: Em Medicos, Pagamentos, TiposAtos e Utentes (2 ocorrências por controlador), as exceções capturadas originam mensagens sobre possível duplicação sem distinguir a causa efetiva.

Impacto: Informação enganadora ao utilizador e diagnóstico dificultado.

Recomendação: Distinguir violações conhecidas de falhas inesperadas e registar erro técnico sem dados sensíveis.

Limite da evidência: Não se demonstrou perda de dados.

AUD-017 — Observabilidade e logging contextual

Classificação: Média | Evidência: Program.cs, appsettings.json e pesquisa de ILogger

Observação: Existem níveis de logging Default=Information e Microsoft.AspNetCore=Warning e tratamento global de exceções; não foram identificadas chamadas explícitas a ILogger/LogError/LogWarning no código C# pesquisado.

Impacto: Dificuldade potencial em relacionar falhas com operações de negócio.

Recomendação: Definir estratégia de registo estruturado e retenção, sem informação clínica desnecessária.

Limite da evidência: A infraestrutura ASP.NET Core pode produzir logs por defeito; ausência total de logging não foi demonstrada.

AUD-018 — Confiança no certificado da ligação SQL

Classificação: Média | Evidência: Inspeção booleana de ConnectionStrings

Observação: A cadeia analisada usa autenticação integrada, sem utilizador ou password explícitos, mas inclui TrustServerCertificate=True. Apenas appsettings.json foi encontrado na raiz do projeto Web.

Impacto: Com encriptação ativa, a cadeia de confiança do certificado do servidor pode não ser validada.

Recomendação: Para produção, usar certificado de servidor confiável e separar configuração por ambiente quando adequado.

Limite da evidência: Configuração pode ser aceitável para demonstração local; não foi observada fuga de credenciais.

AUD-019 — Rota global de erro não localizada

Classificação: Média | Evidência: UseExceptionHandler("/Home/Error") e pesquisas de código/views

Observação: O Program.cs encaminha erros para /Home/Error fora de desenvolvimento; não existe HomeController.cs, nem foram encontrados métodos Error pelas assinaturas pesquisadas ou views Error/Home.

Impacto: Possível falha secundária ao apresentar uma exceção inesperada.

Recomendação: Disponibilizar rota de erro válida e validar comportamento em ambiente equivalente a produção.

Limite da evidência: Confirmação por análise estática; execução do cenário de falha não realizada.

AUD-020 — Proteção antifalsificação em ações POST

Classificação: Boa prática | Evidência: Pesquisa de atributos nos controladores

Observação: 21 ações explicitamente identificadas com [HttpPost] incluem [ValidateAntiForgeryToken]; não foram encontrados atributos HttpPut/HttpPatch/HttpDelete nem IgnoreAntiforgeryToken.

Impacto: Reduz o risco de pedidos falsificados nas ações POST analisadas.

Recomendação: Manter proteção e validar tokens nas views; verificar ações sem verbo explícito que possam alterar dados.

Limite da evidência: Cobertura de atributos, não teste funcional completo.

5. Recomendações priorizadas

Prioridade	Ação recomendada (sem implementação nesta PoC)
P1	Definir requisitos de acesso e autorização para qualquer implantação real; resolver regra de agendamento sem horário.
P1	Triar avisos de dependências NuGet e avaliar atualizações/alcance.
P2	Formalizar regra de remoção de horários com consultas futuras e pagamentos de consultas canceladas.
P2	Garantir rota de erro funcional, logging contextual e tratamento diferenciado de falhas de persistência.
P2	Rever confiança no certificado SQL Server para ambientes não locais.
P3	Planear testes automatizados, concorrência e revisão contextual dos avisos dos analisadores.

6. Conclusão

A PoC demonstrou a complementaridade entre análise automatizada e consultoria técnico-funcional: as ferramentas identificaram avisos de dependências e padrões de código; a análise de regras de negócio e o teste funcional identificaram uma marcação aceite sem horário médico, que não decorre diretamente de um aviso de compilação. O relatório regista igualmente pontos positivos. Não foram efetuadas alterações à solução auditada.

ANEXO A — Comandos de auditoria e resultados

Os comandos abaixo são representativos das verificações efetuadas. São apresentados sem segredos, caminhos pessoais ou resultados repetidos. A saída integral de comandos extensos deverá ser preservada como evidência bruta separada.

EV-001 — Build e analisadores

```
dotnet build Consultorio.slnx --no-restore -t:Rebuild -p:AnalysisMode=All -p:AnalysisLevel=latest -p:EnforceCodeStyleInBuild=true -p:RunAnalyzers=true
```

Resultado observado: Build concluído; 190 avisos (186 CA e 4 ocorrências NuGet).

EV-002 — Dependências

```
dotnet package list --project Consultorio.slnx --include-transitive --no-restore
```

Resultado observado: Listagem de dependências, incluindo SqlClient e Pkcs; avisos NU1902 e NU1903 na compilação.

EV-003 — Exceções DbUpdateException

```
powershell -NoProfile -Command "Get-ChildItem 'Consultorio.Web\Controllers' -Filter '*.cs' | Select-String -Pattern 'catch\s*\(DbUpdateException'"
```

Resultado observado: Oito ocorrências em quatro controladores.

EV-004 — Logging e tratamento de erros

```
powershell -NoProfile -Command "Get-ChildItem 'Consultorio.Web' -Recurse -Filter '*.cs' | Where-Object { $_.FullName -notmatch '\\obj\\|\\bin\\' } | Select-String -Pattern 'ILogger|LogError|LogWarning|UseExceptionHandler|AddLogging'"
```

Resultado observado: Uma ocorrência: UseExceptionHandler no Program.cs.

EV-005 — Nomes de secções de configuração

```
powershell -NoProfile -Command "$j = Get-Content 'Consultorio.Web\appsettings.json' -Raw | ConvertFrom-Json; $j.PSObject.Properties | ForEach-Object { $_.Name }"
```

Resultado observado: ConnectionStrings; Logging; AllowedHosts.

EV-006 — Níveis de logging

```
powershell -NoProfile -Command "$j = Get-Content 'Consultorio.Web\appsettings.json' -Raw | ConvertFrom-Json; $j.Logging | ConvertTo-Json -Depth 5"
```

Resultado observado: Default=Information; Microsoft.AspNetCore=Warning.

EV-007 — Configurações por ambiente

```
powershell -NoProfile -Command "Get-ChildItem 'Consultorio.Web' -File -Filter 'appsettings*.json' | Select-Object -ExpandProperty Name"
```

Resultado observado: Apenas appsettings.json.

EV-008 — Pesquisa de atributos CSRF

```
powershell -NoProfile -Command "Get-ChildItem 'Consultorio.Web\Controllers' -Filter '*.cs' | Select-String -Pattern '\[HttpPost|\[ValidateAntiForgeryToken|\[AutoValidateAntiforgeryToken'"
```

Resultado observado: 21 ações POST, 21 atributos antiforgery.

EV-009 — Outros verbos/dispensas CSRF

```
powershell -NoProfile -Command "Get-ChildItem 'Consultorio.Web\Controllers' -Filter '*.cs' | Select-String -Pattern 'HttpPut|HttpPatch|HttpDelete|IgnoreAntiforgeryToken|AutoValidateAntiforgeryToken'"
```

Resultado observado: Sem ocorrências.

EV-010 — Operações de persistência

```
powershell -NoProfile -Command "Get-ChildItem 'Consultorio.Web\Controllers' -Filter '*.cs' | Select-String -Pattern 'SaveChangesAsync|ExecuteDeleteAsync|ExecuteUpdateAsync|Remove\(|RemoveRange\('"
```

Resultado observado: 21 chamadas SaveChangesAsync; duas remoções explícitas em MedicosController.

EV-011 — Rota de erro

```
powershell -NoProfile -Command "Get-ChildItem 'Consultorio.Web\Views' -Recurse -File | Where-Object { $_.Name -match 'Error|Exception' -or $_.FullName -match '\\Home\\' } | Select-Object -ExpandProperty FullName"
```

Resultado observado: Sem resultados.

EV-012 — Método de remoção

```
powershell -NoProfile -Command "Get-Content 'Consultorio.Web\Controllers\MedicosController.cs' | Select-Object -Skip 270 -First 30"
```

Resultado observado: POST com antiforgery; remove horário sem consulta a agendamentos futuros.

ANEXO B — Evidências de código selecionadas

Program.cs

```
   5: builder.Services.AddDbContext<ConsultorioDbContext>(options =>
options.UseSqlServer(builder.Configuration.GetConnectionString("Consultorio")));
   6: var app = builder.Build();
   7: if (!app.Environment.IsDevelopment()) { app.UseExceptionHandler("/Home/Error"); app.UseHsts(); }
   8: app.UseHttpsRedirection(); app.UseStaticFiles(); app.UseRouting();
   9: app.MapControllerRoute(name:"default",pattern: "{controller=Dashboard}/{action=Index}/{id?}");
  10: app.Run();
```

Controllers/MedicosController.cs

```
 275:     [HttpPost]
 276:     [ValidateAntiForgeryToken]
 277:     public async Task<IActionResult> RemoverHorario(
 278:     int id,
 279:     int medicoId)
 280:     {
 281:         var horario = await db.HorariosMedicos
 282:             .FirstOrDefaultAsync(h =>
 283:                 h.HorarioMedicoId == id &&
 284:                 h.MedicoId == medicoId);
 285:
 286:         if (horario is null)
 287:             return NotFound();
 288:
 289:         db.HorariosMedicos.Remove(horario);
 290:
 291:         await db.SaveChangesAsync();
 292:
 293:         TempData["MensagemSucesso"] =
 294:             "Horário removido com sucesso.";
 295:
 296:         return RedirectToAction(
 297:             nameof(Horarios),
 298:             new { id = medicoId });
 299:     }
```

Controllers/PagamentosController.cs

```
 118:     }
 119:
 120:     private async Task ValidarAsync(PagamentoFormViewModel model, int? id = null)
 121:     {
 122:         if (!await db.Consultas.AnyAsync(c => c.ConsultaId == model.ConsultaId))
 123:             ModelState.AddModelError(nameof(model.ConsultaId), "A consulta selecionada não existe.");
 124:         if (model.DataPagamento > DateTime.Now)
 125:             ModelState.AddModelError(nameof(model.DataPagamento), "A data do pagamento não pode ser
futura.");
 126:         var referencia = model.ReferenciaFicticia?.Trim();
 127:         if (!string.IsNullOrEmpty(referencia) && await db.Pagamentos.AnyAsync(p =>
 128:             p.ReferenciaFicticia == referencia && (!id.HasValue || p.PagamentoId != id.Value)))
 129:             ModelState.AddModelError(nameof(model.ReferenciaFicticia), "Esta referência já está
registada.");
 130:     }
 131:
```

```
132:     private async Task CarregarConsultasAsync()
133:     {
134:         var consultas = await db.Consultas.AsNoTracking()
135:             .Include(c => c.Utente)
136:             .OrderByDescending(c => c.DataHora)
137:             .Select(c => new { c.ConsultaId, c.DataHora, Utente = c.Utente != null ? c.Utente.Nome : "—"
})
138:             .ToListAsync();
139:         ViewBag.Consultas = new SelectList(consultas.Select(c => new
140:         {
141:             c.ConsultaId,
142:             Texto = $"#{c.ConsultaId} · {c.DataHora:dd/MM/yyyy HH:mm} · {c.Utente}"
```

ANEXO C — Registo de evidências externas e limites

TF-001: teste funcional de agendamento sem horário médico, consulta fictícia #2005, 22/10/2026 10:15–10:45. A captura de ecrã original deverá ser anexada à versão final caso seja selecionada e validada.

Saída integral dos 190 avisos .NET: preservar ficheiro de consola como evidência bruta; a matriz acima apresenta contagens consolidadas, não transcrição integral de 190 linhas.

Os resultados de pesquisa com saída vazia significam “não encontrado no âmbito/padrão pesquisado”; não constituem prova universal de inexistência.